

GÉNÉRATION DE CODE MACHINE (ASSEMBLEUR 8086)

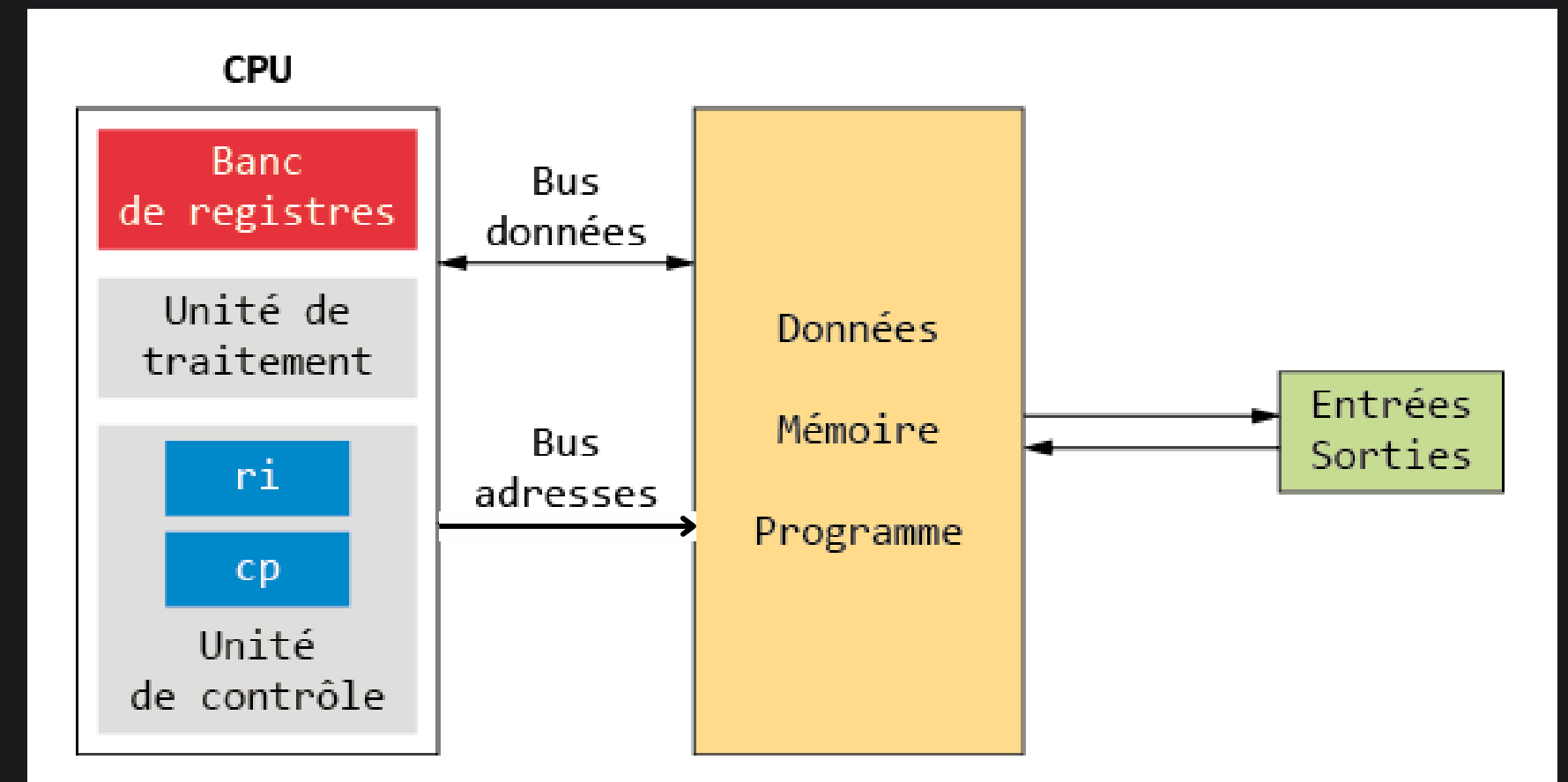
Génération de code machine (assembleur 8086)

- La génération de code machine pour l'assembleur 8086 se situe dans la **dernière phase** du processus de compilation.
- Elle consiste à **traduire** le **code intermédiaire** en instructions assembleur spécifiques à l'architecture Intel 8086.

Rappel: Niveaux de programmation

Circuits logiques (0/1)

- L'UAL est intégrée dans le processeur, et ses opérations sont effectuées en binaire, manipulant des données sous forme de 0 et 1.
- L'Unité de Commande envoie des signaux de contrôle pour exécuter les instructions sous forme de micro-instructions.



- Les circuits logiques jouent un rôle central dans l'exécution des instructions sur le processeur.

Rappel: Niveaux de programmation

Codop

- C'est la **représentation binaire** d'une **instruction machine**.
- Un programme en langage machine est une suite de bits (0 et 1) que le processeur peut exécuter directement.
- Exemple: 11111 000011101010101 peut correspondre à une instruction machine dans une architecture spécifique.

Langage assembleur

- Il utilise des **mnémoniques** pour représenter les instructions machines sous une forme plus lisible.
- Ce sont des représentations textuelles des instructions machine, plus lisibles pour les humains. Par exemple, ADD (addition), MOV (déplacement), JZ (saut si zéro), etc.

Structure d'un programme assembleur 8086

TITLE nom ; Définir le nom du programme

PILE **SEGMENT STACK** ; Déclaration d'un segment de pile nommé "PILE"

.....

.....

PILE **ENDS** ; Fin de la déclaration de la pile

DONNEE **SEGMENT** ; Déclaration du segment de données

.....

.....

DONNEE **ENDS** ; Fin de la déclaration de données

LECODE **SEGMENT** ; Déclaration du segment de code qui va contenir le code

Debut: ; etiquette indiquant le point d'entrée du programme

.....

.....

LECODE **ENDS** ; Fin de la déclaration de code

; ----- Fin du Programme -----

END Debut ; Indiquer la fin du point d'entrée du code

Extension

Vos programmes compilés doivent avoir l'extension :

.asm pour **AS**se**M**bler

Quelques registres les plus utilisés

- Les **registres** sont des **emplacements de mémoire** très rapides situés directement dans le processeur.
- Ils sont utilisés pour stocker **temporairement** des données ou des résultats d'opérations.
- Dans l'assembleur 8086, on distingue plusieurs types de registres :

1. Registres généraux

- AX (Accumulateur) : Utilisé pour les calculs.
- BX (Base) : utilisé pour accéder à la mémoire
- CX (Compteur) : Utilisé comme compteur dans les boucles, par exemple pour répéter une opération un certain nombre de fois.
- DX (Data Extension Register): est un registre polyvalent, mais a un rôle spécial dans les opérations de multiplication et division.

2. Registres de la pile

- SP (Stack Pointer – Pointeur de pile) : Il pointe toujours vers le **sommet actuel de la pile**. Son contenu est **automatiquement** changé par les instructions **PUSH** et **POP**.
- BP (Base Pointer) : sert de pointeur sur **la base de la pile**, et permet d'atteindre n'importe quel élément de la pile (exemple: `MOV AX,[BP+2]`). Contrairement à SP, **BP** est souvent manipulé **manuellement** pour naviguer dans la pile.

3. Registres de segment

Les registres de segment servent à indiquer au processeur où se trouvent les différentes parties du programme en mémoire.

- DS (Data Segment): Pointe sur le début du segment qui contient les données.
- CS (Code Segment): Pointe sur le segment contenant le code du programme.
- SS (Stack Segment): segment contenant la pile.
- ES (Extra Segment): segment supplémentaire.

Format standard d'une instruction

Les instructions sont les commandes que le processeur exécute. Le format standard d'une instruction est le suivant:

Label: Mnémonique Opérands ;commentaire

Qu'est-ce qu'un label ?

- Les labels (étiquettes) sont des noms donnés à des lignes du programme.
- Ils servent de **repères** pour les instructions de saut (conditionnels ou non) et permettent de pointer vers une instruction précise.

Exemple :

START: ;label de début

→ Les labels servent à rendre le code lisible et à contrôler le flux d'exécution.

Qu'est-ce qu'un Mnémonique ?

Il sert à identifier une instruction donnée. Par exemple:

- MOV = move (copy)
- ADD = addition
- JMP = jump
- CMP = compare
- INT = interrupt

Qu'est-ce qu'un opérande?

- Un opérande est une donnée utilisée par une instruction (peut-être un registre, valeur ou mémoire).
- En 8086, une instruction peut avoir **0**, **1** ou **2** opérandes.

Commentaire

- Tout ce qui suit ; sur la même ligne est considéré comme un commentaire.

Quelques instructions

1. Affectations

- Registres ← Valeurs : On place une valeur constante directement dans un registre.

```
MOV AX, 65535    ; Met la valeur décimale 65535 dans AX  
MOV CL, 01101b  ; Met la valeur binaire 01101 (13 décimal) dans CL  
MOV DH, 0FAh    ; Met la valeur hexadécimale FA (250 décimal) dans DH
```

- Entre registres : On copie la valeur d'un registre dans un autre.

```
MOV AX, BX    ; AX reçoit la valeur de BX  
MOV CL, DH    ; CL reçoit la valeur de DH
```

- Registres ↔ Variables mémoire: On lit ou écrit une valeur en mémoire (variable) via les registres.

```
MOV CX, variable_de_deux_octets    ; CX reçoit la valeur de la variable (2 octets)  
MOV variable_de_un_octet, DL      ; la variable reçoit la valeur de DL
```

- Registres ← Adresses mémoire

```
MOV AX, OFFSET variable ; AX reçoit l'adresse mémoire de la variable
MOV CX, [5Ah] ; CX reçoit la valeur stockée à l'adresse 5Ah
```

- OFFSET : L'offset est la position d'une donnée à l'intérieur d'un segment mémoire, pas sa valeur.
- [5Ah] : accède à la valeur contenue à l'adresse mémoire 5A (accès direct)

2. Arithmétique

Ces instructions permettent de faire des opérations de base : addition, soustraction, incrémentation, décrémentation.

- **Incrémentation (INC)** : Elle augmente de 1 la valeur d'un registre ou d'une variable mémoire.

```
INC AX ; AX ← AX + 1
INC ma_variable ; ma_variable ← ma_variable + 1
```

- Ne prend qu'un **seul opérande**.

- **Décrémentation (DEC):** Elle diminue de 1 la valeur d'un registre ou d'une variable.

```
DEC AX ; AX ← AX - 1  
DEC ma_variable ; ma_variable ← ma_variable - 1
```

- C'est une instruction unaire.
- **Addition (ADD):** Effectue une addition classique de deux opérandes.

```
ADD AX, 5 ; AX ← AX + 5 (constante immédiate)  
ADD BH, toto ; BH ← BH + valeur de la variable toto  
ADD toto, CX ; toto ← toto + contenu de CX
```

- **Syntaxe :** **ADD destination, source** ; Le résultat est stocké dans l'opérande destination
- **Soustraction SUB:** Effectue une soustraction entre deux opérandes.

```
SUB AX, 5 ; AX ← AX - 5  
SUB BH, toto ; BH ← BH - valeur de la variable toto  
SUB toto, CX ; toto ← toto - CX
```

- **Syntaxe :** **SUB destination, source**

3. Comparaison

Toutes les comparaisons se font à l'aide de l'instruction **CMP**

- **L'instruction CMP (Compare)**: CMP compare deux opérandes en effectuant une soustraction implicite.

CMP A, B ; Internement fait $A - B$

- Mais le résultat n'est PAS stocké dans A. L'opération sert uniquement à mettre à jour les flags (dans le registre **FLAGS**).
- Après un CMP, le processeur regarde certains drapeaux (flags) pour décider du saut :
 - ZF (Zero Flag) : devient 1 si le résultat = 0.
 - SF (Sign Flag) : indique le signe du résultat (0 = positif, 1 = négatif).
 - CF (Carry Flag) : Sous-dépassement ($A < B$), indique une retenue (ou emprunt).
 - OF (Overflow Flag) : Résultat hors limites.

4. Sauts conditionnels (Jump if...)

Après un CMP, on peut utiliser un saut conditionnel (jump) pour rediriger le flux d'exécution selon le résultat.

- Il faut définir des labels (étiquettes): les endroits dans le programme où va sauter si le test est vérifié.

JZ	Saut si zéro.	ZF = 1	JNZ
JE	Saut si égal.		JNE
JC	Saut si Retenue (inférieur).	CF = 1	JNC
JB	Saut si inférieur.		JNB
JNAE	Saut si ni supérieur ni égal.		JAЕ
JS	Saut si signe négatif.	SF = 1	JNS
JO	Saut si débordement.	OF = 1	JNO
JPE	Saut si parité paire.	PF = 1	JPO
JP	Saut si parité.		
JNZ	Saut si pas zéro.	ZF = 0	JZ
JNE	Saut si différent.		JE
JNC	Saut si pas de retenue.	CF = 0	JC
JNB	Saut si pas inférieur.		JB
JAЕ	Saut si supérieur ou égal.		JNAE
JNS	Saut si aucun signe (positif).	SF = 0	JS
JNO	Saut si pas de débordement.	OF = 0	JO
JPO	Saut si parité impaire.	PF = 0	JPE
JNP	Saut si pas de parité.		JP

Exemple:

; Egalité (JE : Jump if Equal)

CMP AX, 5

JE LABEL_1 ; SI $AX == 5 \rightarrow$ SAUT À LABEL_1

; Différence (JNE : Jump If Not Equal)

CMP AX, MA_VARIABLE

JNE LABEL_2 ; SI $AX \neq MA_VARIABLE \rightarrow$ SAUT À LABEL_2

;Inférieur, Supérieur, Inf. ou égal, Sup. ou égal

;(Jump if Lower, Greater, Lower or Equal, Greater or Equal)

CMP CH, 0

JL LABEL_3 ; SI $CH < 0 \rightarrow$ SAUT À LABEL_3 (VALEURS SIGNÉES)

CMP DH, 0AH

JG LABEL_4 ; SI $DH > 10$ EN HEXADÉCIMAL $\rightarrow$ SAUT À LABEL_4

CMP AL, 01001B

JLE LABEL_5 ; SI $AL \leq 9 \rightarrow$ SAUT À LABEL_5

CMP VARIABLE, 65

JGE LABEL_6 ; SI $VARIABLE \geq 65 \rightarrow$ SAUT À LABEL_6

Label_1: instructions...

Label_2: instructions...

Label_3: instructions...

Label_4: instructions...

; Saut non conditionnel

JMP label_1

Récapitulatif de l'instruction MOV – Transfert de données

Syntaxe	Description	Exemple
MOV reg, reg	Transfert entre deux registres	MOV AX, BX
MOV reg, mem	Depuis la mémoire vers un registre	MOV AX, [1234h]
MOV mem, reg	Depuis un registre vers la mémoire	MOV [1234h], AX
MOV reg, immedi	Depuis une constante vers un registre	MOV CX, 10
MOV mem, immedi	Depuis une constante vers la mémoire	MOV toto, 0Ah

Attention !!!!!

Pas de mémoire à mémoire directe



Exemple interdit !!!!

MOV [1234h], [5678h]

→ Ce type de transfert doit être fait en deux étapes via un registre



MOV AX, [5678h] ; charger dans un registre

MOV [1234h], AX ; transférer à l'autre adresse

Trois parties du code assembleur

Un programme assembleur typique est structuré en trois parties : Pile, Données, et code.

1. Segment de pile(Stack Segment)

- **Qu'est-ce qu'une pile ?**

- C'est une zone mémoire spéciale utilisée pour sauvegarder temporairement des données (comme les appels de fonctions et les registres).
- Elle fonctionne selon le principe LIFO (Last In, First Out).

- **Instructions liées à la pile**

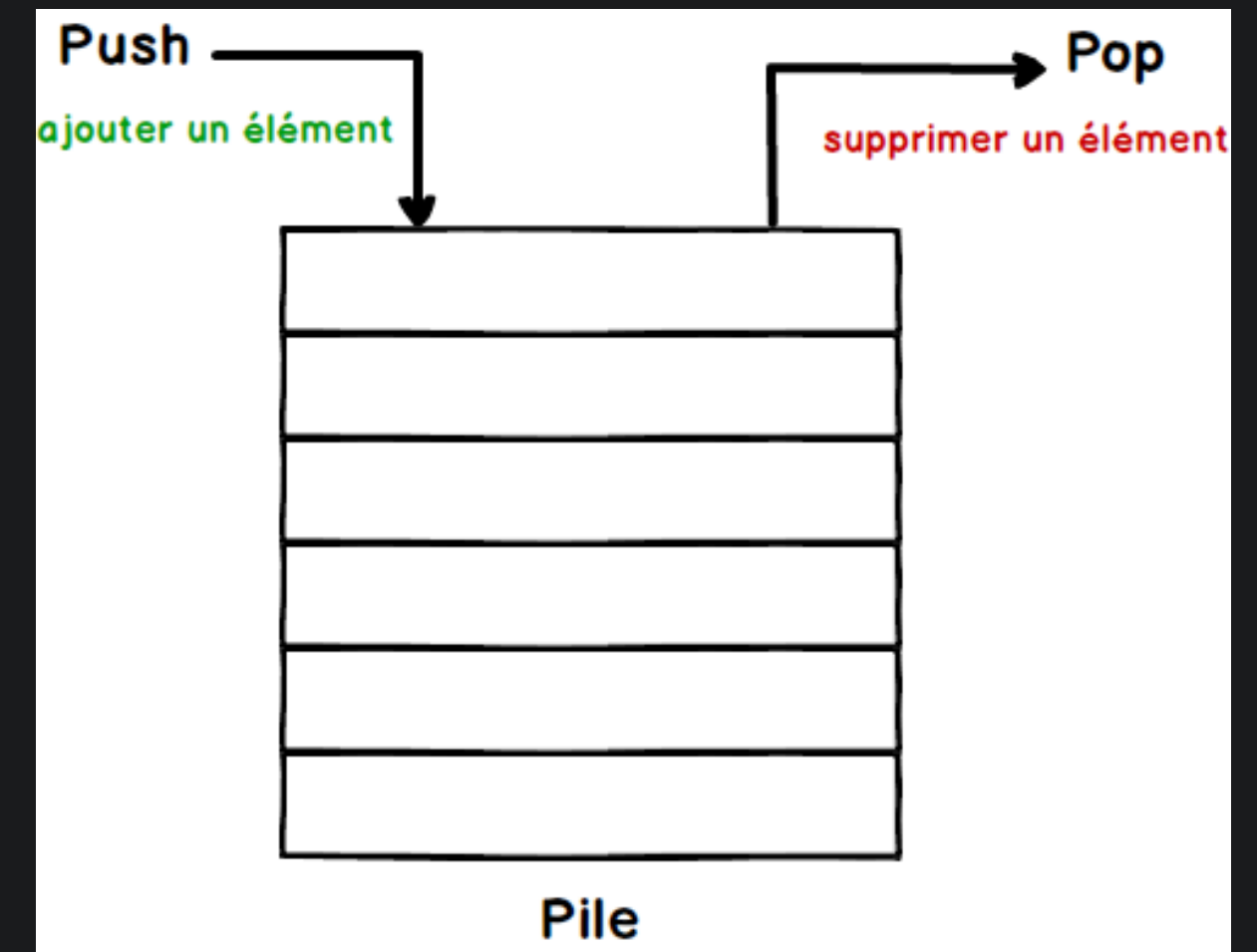
- **PUSH registre**

- Sauvegarde le contenu d'un registre sur la pile.
- Exemple: PUSH AX ; Empile le contenu du registre AX

- **POP registre**

- Restaure une valeur depuis la pile vers un registre.
- Exemple: POP AX ; Dépile contenu du sommet de la pile et le transfère dans

AX



Utilisation de la pile sur un exemple:

PUSH AX ; Sauvegarder la valeur actuelle de AX sur la pile

PUSH BX ; Sauvegarder la valeur actuelle de BX sur la pile

MOV AX, [0140] ; Charger en AX la valeur contenue à l'adresse mémoire 0140

ADD BX, AX ; Ajouter cette valeur à BX (BX = BX + [0140])

MOV [0140], BX ; Stocker le nouveau contenu de BX dans l'adresse 0140

POP BX ; Restaurer l'ancienne valeur de BX depuis la pile

POP AX ; Restaurer l'ancienne valeur de AX depuis la pile

→ Cela permet de faire des opérations en utilisant AX et BX sans perdre leurs valeurs initiales.

Déclaration d'une pile (Stack Segment)

Avant de pouvoir utiliser les instructions PUSH et POP, il faut :

- Réserver un espace mémoire (pile).
- Définir un segment de pile grâce à la directive SEGMENT stack.
- Initialiser les registres SS (Segment Stack) et SP (Stack Pointer)

Structure de déclaration d'une pile

```
segment_pile SEGMENT stack ; Début du segment de pile (mot-clé `stack`)  
    DW 100 DUP (?)          ; Réserve 100 mots = 200 octets non initialisés  
base_pile EQU $           ; Étiquette pour repérer la base de la pile  
                        ;(c'est-à-dire l'adresse juste après la fin de la zone réservée)  
segment_pile ENDS         ; Fin du segment de pile
```

→ Quand la pile est vide, SP doit pointer sous la zone réservée, c'est-à-dire au "haut" de la pile → c'est ce qu'on appelle ici la base de la pile.

Déclaration d'une pile (Stack Segment)

Initialisation des registres pour utiliser la pile

Après la déclaration, on doit initialiser la pile comme suit :

```
ASSUME SS:segment_pile ; Informe l'assembleur du segment à utiliser pour la pile
```

```
MOV AX, segment_pile ; Charge le segment de pile dans AX
```

```
MOV SS, AX ; Initialise SS (segment de pile)
```

```
MOV SP, base_pile ; Pointeur SP positionné en bas de la pile (pile vide)
```

Remarques:

- On **ne peut pas** faire **MOV SS**, segment_pile directement, car ce **n'est pas autorisé**.
- On doit passer par un registre (souvent AX) pour transférer une valeur dans SS.

2. Partie Données (Data Segment)

La section des données est utilisée pour déclarer les variables et les constantes. Elle sert à réserver de la mémoire pour stocker des données utilisées dans le programme.

```
DATA SEGMENT  
  
    ; déclarations ici  
  
DATA ENDS
```

2.2. Déclaration des variables

Syntaxe:

```
nom_variable      type      valeur
```

Exemples:

datas1 DB ? ; 1 octet non initialisé

datas2 DB 255 ; 1 octet initialisé à 255

datas3 DW ? ; 2 octets non initialisés

datas4 DB 5 DUP(?) ; tableau de 5 octets non initialisés

datas5 DW 10 DUP(15) ; tableau de 10 mots initialisés à 15

Types de données:

Type	Nom	Taille	Signification
DB	Declare Byte	1 octet (8 bits)	Ex : DB 100
DW	Declare Word	2 octets (16 bits)	Ex : DW 1234
DD	Declare Double Word	4 octets (32 bits)	Ex : DD 100000
DQ	Declare Quad Word	8 octets (64 bits)	
DF/DP	Declare Float	6 octets (48 bits)	
DT	Declare Ten Bytes	10 octets (80 bits)	

2.2. Représentation des constantes

- **Décimale (base 10)**

- Écriture simple, sans suffixe.
- Exemples : 0, 15, 123

- **Hexadécimale (base 16)**

- Suffixe : h
- Les lettres vont de A à F (équivalent de 10 à 15).
- Si le nombre commence par une lettre, on ajoute un 0 devant.
- Exemple : 0Ah, 0Fh, 1B2Ch

- **Binaire (base 2)**

- Suffixe : b
- Composé uniquement de 0 et 1.
- Exemple : 1010b, 0001b, 11110101b

2.3. Déclaration des vecteurs (tableaux)

Les vecteurs permettent de stocker une **série d'éléments** du **même type**.

- **Vecteurs d'entiers**

- Chaque élément peut être sur un byte ou un word.
- Exemple:

```
T DW 1, 4, 7, -1 ; vecteur de 4 entiers (2 octets chacun) initialisés  
respectivement
```

```
; aux valeurs 1, 4, 7 et -1
```

```
T1 DW 100 DUP(?) ; vecteur de 100 entiers non initialisés
```

- **Accès aux éléments d'un vecteur**

Puisque chaque élément occupe 2 octets (DW) dans l'exemple précédent, il faut multiplier l'indice par 2 pour accéder au bon emplacement mémoire.

```
MOV SI, i    ; i est l'indice  
ADD SI, SI    ; SI = i * 2  
MOV AX, T[SI] ; AX <-- T[i]
```

```
MOV SI, i  
ADD SI, SI  
MOV T[SI], AX ; T[i] <-- AX
```

3. Partie Code (Code Segment)

Cette partie contient le programme exécutable, les instructions qui sont traitées par le processeur. C'est ici que se trouve la logique de votre programme, les calculs, et les appels système.

CODE SEGMENT

START:

MOV AX, 10 ; Charger 10 dans AX

ADD AX, 5 ; Ajouter 5 à AX

MOV BX, AX ; Copier AX dans BX

RET ; Retour de la fonction

CODE ENDS

Ici, START est un label qui représente le début du programme, et les instructions MOV, ADD, RET sont exécutées dans cet ordre.

Modes d'adressages

1. Adressage immédiat (Immediate)

- L'opérande est une valeur constante donnée directement dans l'instruction.
- Il est utilisé pour : affecter une valeur constante à un registre ou une variable.

```
MOV AX, 5 ; AX ← 5
```

```
MOV BL, 0Fh ; BL ← 0Fh
```

2. Adressage direct (Direct)

- L'opérande est situé à une adresse mémoire explicite.
- il est utilisé pour : accéder directement à une case mémoire précise.

```
mov [0C040h], AL ; mémoire à l'adresse 0C040h ← AL
```

```
mov DS:[0C040h], AL ; même opération, segment DS précisé
```

Modes d'adressages

3. Mode d'adressage par registre

- Le mode registre signifie que les deux opérandes sont des registres.
- La donnée est copiée directement d'un registre vers un autre. Utilisé pour des transferts ou calculs rapides entre registres.

```
mov AX, BX ; AX ← contenu de BX
```

```
MOV CL, DL ; CL ← contenu de DL
```

Exemple sur ROL:

TITLE prog2.asm

Pile segment stack ;

dw 100 dup(?)

Pile ends

DATA segment

DATA1 DW 5F97H

COMPTE DB ?

DATA ends

CODE segment

MAIN: ASSUME CS:CODE, DS:DATA

MOV AX, DATA

MOV DS, AX

XOR BL, BL ; Mettre BL à 0 (ou bien SUB)

MOV DL, 16 ; rotation 16 fois

MOV AX, DATA1

ENCORE: ROL AX, 1 ; Rotation a gauche (a droite aussi si on préfère)

JNC PROCHAIN ; Test si CF=0

INC BL ; Si CF = 1, incrémenter le compteur du nombre de '1'

PROCHAIN: DEC DL ; répéter 16 fois

JNZ ENCORE ; encore une fois si ce n'est pas fini

MOV COMPTE, BL ; sauvegarder le résultat dans la case mémoire

COMPTE

MOV AH, 4Ch

INT 21h

MAIN ENDS

END MAIN

L'équivalent de quelques instructions du langage C en assembleur

1. Équivalent if / else en assembleur

```
if (AX == 1) {  
    BX = 10;  
} else {  
    BX = 0;  
}  
CX = 10;
```

if: CMP AX, 1 ; Compare AX avec 1
JNZ Else ; Si AX $\neq$ 1, sauter à l'étiquette Else

Then: ; Partie "if"
MOV BX, 10 ; BX $\leftarrow$ 10
JMP EndIf ; Saut inconditionnel vers la fin

Else: ; Partie "else"
MOV BX, 0 ; BX $\leftarrow$ 0

EndIf: ; Suite du programme
MOV CX, 10 ; CX $\leftarrow$ 10

2. Boucle for en Assembleur

```
BX = 0;  
for (k = 0; k <= 10; k++) {  
    BX = BX + k;  
}
```

MOV BX, 0 ; Initialisation de BX à 0 → BX = BX + k à chaque itération

MOV CX, 0 ; Initialisation de k (ici représenté par CX)

For: CMP CX, 10 ; Test : si CX > 10 → fin de la boucle

JA EndFor ; JA = Jump if Above (utilisé ici comme CX > 10 → sortir)

ADD BX, CX ; BX = BX + CX (on ajoute la valeur courante de k)

INC CX ; k++ (incrémentatation)

JMP For ; Retour au début de la boucle

EndFor: ; Étiquette de fin de boucle

2. Équivalent Switch case en Assembleur

```
switch(n) {  
    case 1:  
        // instructions...  
        break;  
    case 2:  
        // instructions...  
        break;  
    default:  
        // instructions...  
}
```

```
CMP n, 1      ; Comparer n à 1  
JNE case2     ; Si n ≠ 1, aller à case2  
  
; ===== CASE 1 =====  
; instructions du case 1  
JMP endswitch ; Aller à la fin du switch  
  
case2:  
CMP n, 2      ; Comparer n à 2  
JNE default   ; Si n ≠ 2, aller au default  
  
; ===== CASE 2 =====  
; instructions du case 2  
JMP endswitch ; Aller à la fin du switch  
  
default:  
; ===== DEFAULT =====  
; instructions par défaut  
endswitch:  
; ===== FIN DU SWITCH =====
```

**Merci pour votre
attention**

Bon courage ...